

Как работает Web

Прочитать URL и HTTP-обмен: увидеть метод, путь, статус и понять, зачем нужен HTTPS.

DAY 04

URL

HTTP

HTTPS

request

response

ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

Конкретные действия вместо списка терминов.

- 01 Прочитать URL и HTTP-обмен: увидеть метод, путь, статус и понять, зачем нужен HTTPS.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример кода и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.

ВЧЕРА

Интернет, клиент и сервер

Что уже есть в нашей модели.



СЕГОДНЯ

Как работает Web

Вчера: Интернет, клиент и сервер. Сегодня мы добавляем следующий слой: как работает web.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

Web — это не «страницы в браузере», а понятный цикл запрос → обработка → ответ. Эта модель понадобится для Flask и API.

Вопрос себе

Какую реальную проблему решает сегодняшняя тема?

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

HTTP похож на заказ в ресторане: метод — что просим сделать, путь — какое блюдо, заголовки — дополнительные условия, тело — данные заказа, статус — итог кухни.

ПЕРЕВОД В IT

URL

полный адрес ресурса в интернете

HTTP

протокол обмена данными между веб-клиентом и сервером

HTTPS

защищённый шифрованием вариант HTTP

Слой 1: URL • HTTP • HTTPS

Сначала смысл, затем профессиональный термин.

**URL**

полный адрес ресурса в интернете

**HTTP**

протокол обмена данными между веб-клиентом и сервером

**HTTPS**

защищённый шифрованием вариант HTTP

Слой 2: request • response • status code

Сначала смысл, затем профессиональный термин.

**request**

сообщение клиента с просьбой выполнить действие

**response**

ответ сервера со статусом и данными

**status code**

числовой результат обработки HTTP-запроса

Слой 3: HTTP method • header

Сначала смысл, затем профессиональный термин.

HTTP method

действие запроса, например GET или POST

header

служебная информация запроса или ответа

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: body • TLS

Сначала смысл, затем профессиональный термин.

body

основные данные, передаваемые в запросе или ответе

TLS

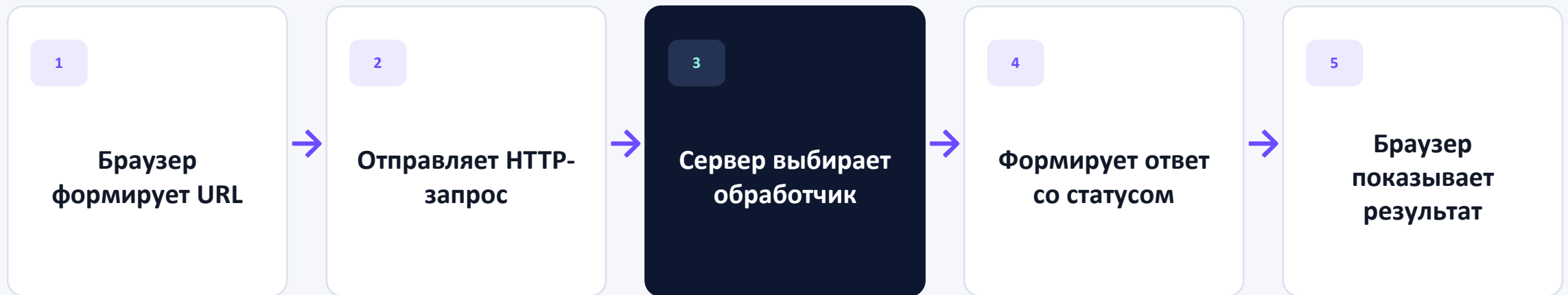
протокол шифрования защищённого сетевого соединения

Не заучивай формулировку. Придумай свой пример использования каждого термина.

СХЕМА

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.



day04_step1

```
method = "GET"  
path = "/courses/1"  
status = 200  
  
print(method, path)  
print("status:", status)
```

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

```
method = "GET"
```

Сохраняем значение или результат вычисления под понятным именем.

```
path = "/courses/1"
```

Сохраняем значение или результат вычисления под понятным именем.

```
status = 200
```

Сохраняем значение или результат вычисления под понятным именем.

```
print(method, path)
```

Показываем результат, чтобы увидеть состояние программы.

```
print("status:", status)
```

Показываем результат, чтобы увидеть состояние программы.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
method = "GET"  
path = "/courses/1"  
status = 200  
  
print(method, path)  
print("status:", status)
```

РЕЗУЛЬТАТ

```
GET /courses/1  
status: 200
```

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.



changed

```
url = "https://example.com/profile"  
method = "GET"  
print(method, url)
```



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

GET https://example.com/profile

ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.

```
url = https://example.com/profile  
print(url)
```

wrong

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

URL — текст. Без кавычек Python пытается разобрать его как код и получает синтаксическую ошибку.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

ИСПРАВЛЕНИЕ

Исправленный вариант

Меняем только то, что действительно было причиной.

```
url = "https://example.com/profile"  
print(url)
```

fixed

ПРАВИЛО

Не переписывай всё решение. Найди минимальное осмысленное исправление.

Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Разобрать запрос GET /courses/1 и ответ 200 с телом «Python Basics».

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 **Клиент выбирает метод GET.**
- 2 Путь /courses/1 указывает ресурс.
- 3 Сервер находит курс с id=1.
- 4 Возвращает статус 200.
- 5 Тело ответа содержит данные курса.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.

```
method = "GET"  
path = "/courses/1"  
status = 200  
body = "Python Basics"
```

solution_part1

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.



```
method = "GET"  
path = "/courses/1"  
status = 200  
body = "Python Basics"  
  
print(method, path)  
print(status, body)
```

solution_final

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

GET /courses/1
200 Python Basics

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобрannую — но теперь первый ход за тобой.

Что означает GET /profile → 404? Объясни отдельно метод, путь и статус.

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

404 говорит о результате поиска ресурса, а не о «сломавшемся интернете».

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



practice_solution

```
method = "GET"  
path = "/profile"  
status = 404  
print(method, path, "->", status)
```

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

GET /profile -> 404

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую закончённую задачу.

ЗАДАНИЕ

Собери три ответа сервера: 200, 404 и 500 — и подпиши их смысл.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.



```
statuses = {200: "OK", 404: "Not Found", 500: "Server Error"}  
for code, meaning in statuses.items():  
    print(code, meaning)
```

mini_project

РЕЗУЛЬТАТ ПРОЕКТА

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

200 OK
404 Not Found
500 Server Error

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01

Считать URL и домен одним и тем же.

02

Путать HTTP-метод со статусом.

03

Думать, что HTTPS меняет смысл методов.

04

Интерпретировать любой 4xx как поломку сервера.

МИНИ-ТЕСТ

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

1

Что такое «URL» и какую проблему это понятие помогает описать?

2

Чем «URL» отличается от «request»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 URL — полный адрес ресурса в интернете.

2 URL: полный адрес ресурса в интернете. request: сообщение клиента с просьбой выполнить действие.

ОТВЕТЫ

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3 **GET /courses/1**
status: 200

4 **URL — текст. Без кавычек Python пытается разобрать его как код и получает синтаксическую ошибку.**

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ Прочитать URL и HTTP-обмен: увидеть метод, путь, статус и понять, зачем нужен HTTPS.
- ✓ Читать и объяснять пример темы «Как работает Web».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

ДАЛЬШЕ

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

Как создают программы в команде

Завтра: Как создают программы в команде. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.